Distributed Azure Training – Lane Detection

NetApp Solutions

NetApp
March 19, 2024

This PDF was generated from https://docs.netapp.com/de-de/netapp-solutions/ai/runai-ld_solution_overview.html on March 19, 2024. Always check docs.netapp.com for the latest.

Inhalt

TR-4896: Distributed Training in Azure: Lane Detection - Lösungsdesign . . . 1
Lösungsüberblick . . . 1
Lane-Erkennung – verteiltes Training mit RUN:AI . . . 4
Schlussfolgerung . . . 29
Weitere Informationen . . . 29

TR-4896: Distributed Training in Azure: Lane Detection - Lösungsdesign

Muneer Ahmad und Verron Martina, NetApp Ronen dar, RUN:AI

Seit Mai 2019 bietet Microsoft einen nativen Portal-Service auf Basis von Azure für NFS- und SMB-Fileservices der Enterprise-Klasse basierend auf NetApp ONTAP Technologie. Ziel dieser Entwicklung ist eine strategische Partnerschaft zwischen Microsoft und NetApp, mit der auch die Ausweitung der Reichweite erstklassiger ONTAP-Datenservices auf Azure möglich ist.

NetApp, ein führender Anbieter von Cloud-Datenservices, hat sich zusammengeschlossen: KI, ein Unternehmen, das KI-Infrastruktur virtualisiert, ermöglicht schnellere KI-Experimente mit voller GPU-Auslastung. Dank dieser Partnerschaft können Teams die KI beschleunigen, indem sie zahlreiche Experimente parallel ausführen und dabei schnell auf Daten zugreifen – und dabei grenzenlose Computing-Ressourcen nutzen. RUN: KI ermöglicht volle GPU-Auslastung durch automatisiertes Ressourcenzuweisung. Mit der bewährten Architektur von Azure NetApp Files kann jedes Experiment mit maximaler Geschwindigkeit ausgeführt werden, indem die Hindernisse für die Datenpipeline beseitigt werden.

NetApp und RUN: KI haben gemeinsam ihre Kräfte vereint und bieten Kunden eine zukunftssichere Plattform für ihren KI-Einsatz in Azure. Von Analysen und High-Performance-Computing (HPC) bis hin zu autonomen Entscheidungen (bei denen Kunden ihre IT-Investitionen optimieren können, indem sie nur das zahlen, was sie benötigen, und zwar zum richtigen Zeitpunkt), bietet die Allianz zwischen NetApp und RUN: KI ermöglicht ein einheitliches Arbeiten in der Azure Cloud.

Lösungsüberblick

In dieser Architektur liegt der Schwerpunkt auf dem rechenintensivsten Teil des verteilten Trainingsprozesses von Lane-Erkennung durch KI oder Machine Learning (ML). Fahrspurerkennung ist eine der wichtigsten Aufgaben im autonomen Fahren, die hilft, Fahrzeuge durch Lokalisierung der Fahrspurmarkierungen zu führen. Statische Bauteile wie Fahrspurmarkierungen führen das Fahrzeug dazu, interaktiv und sicher auf der Autobahn zu fahren.

Convolutional Neural Network (CNN)-basierte Ansätze haben das Verständnis und die Segmentierung von Szenen auf eine neue Ebene verschoben. Auch wenn es nicht gut funktioniert für Objekte mit langen Strukturen und Regionen, die verdeckt werden könnten (z. B. Pole, Schatten auf der Spur, und so weiter). Das Spatial Convolutional Neural Network (SNN) verallgemeinert die CNN auf eine reiche räumliche Ebene. Es ermöglicht die Informationsverbreitung zwischen Neuronen in der gleichen Schicht, was es am besten für strukturierte Objekte wie Lanes, Pole oder LKW mit Oktazionen geeignet macht. Diese Kompatibilität ist, weil die räumliche Information verstärkt werden kann, und es bewahrt Glätte und Kontinuität.

Tausende von Szenenbildern müssen im System injiziert werden, damit das Modell die verschiedenen Komponenten im Datensatz erlernen und unterscheiden kann. Diese Bilder umfassen Wetter, tagsüber oder nachts, Mehrspurstraßen und andere Verkehrsbedingungen.

Aus dem Training werden gute Qualität und Quantität der Daten benötigt. Eine einzelne GPU oder mehrere GPUs können Tage bis Wochen dauern, bis das Training abgeschlossen ist. Das Training mit verteilten Daten kann den Prozess mithilfe von GPUs mit mehreren Nodes beschleunigen. Horovod ist ein solches Framework, das verteiltes Training gewährt, aber das Lesen von Daten über GPUs Cluster hinweg kann als Hindernis

fungieren. Azure NetApp Files bietet einen extrem hohen Durchsatz und kontinuierlich niedrige Latenz, um Funktionen für die horizontale und vertikale Skalierung bereitzustellen, sodass GPUs optimal auf die Computing-Kapazität eingesetzt werden. Unsere Experimente bestätigten, dass alle GPUs im Cluster durchschnittlich mehr als 96 % beim Training der Lane-Erkennung mit SNN verwendet werden.

Zielgruppe

Data Science beinhaltet mehrere Disziplinen in IT und Business, weshalb mehrere Personas Teil unserer Zielgruppe sind:

- Die Data Scientists benötigen die Flexibilität, die Tools und Bibliotheken ihrer Wahl einzusetzen.
- Data Engineers müssen wissen, wie die Daten fließen und wo sie sich befinden.
- Autonomes Fahren – Fallexperten.
- Cloud-Administratoren und -Architekten, um Cloud-Ressourcen (Azure) einzurichten und zu managen
- DevOps-Engineers benötigen die Tools, um neue KI/ML-Applikationen in ihre Continuous Integration und Continuous Deployment (CI/CD)-Pipelines zu integrieren.
- Geschäftsanwender möchten auf KI-/ML-Applikationen zugreifen können.

In diesem Dokument beschreiben wir, wie Azure NetApp Files, RUN: AI und Microsoft Azure jeder dieser Rollen einen Mehrwert für das Unternehmen bieten können.

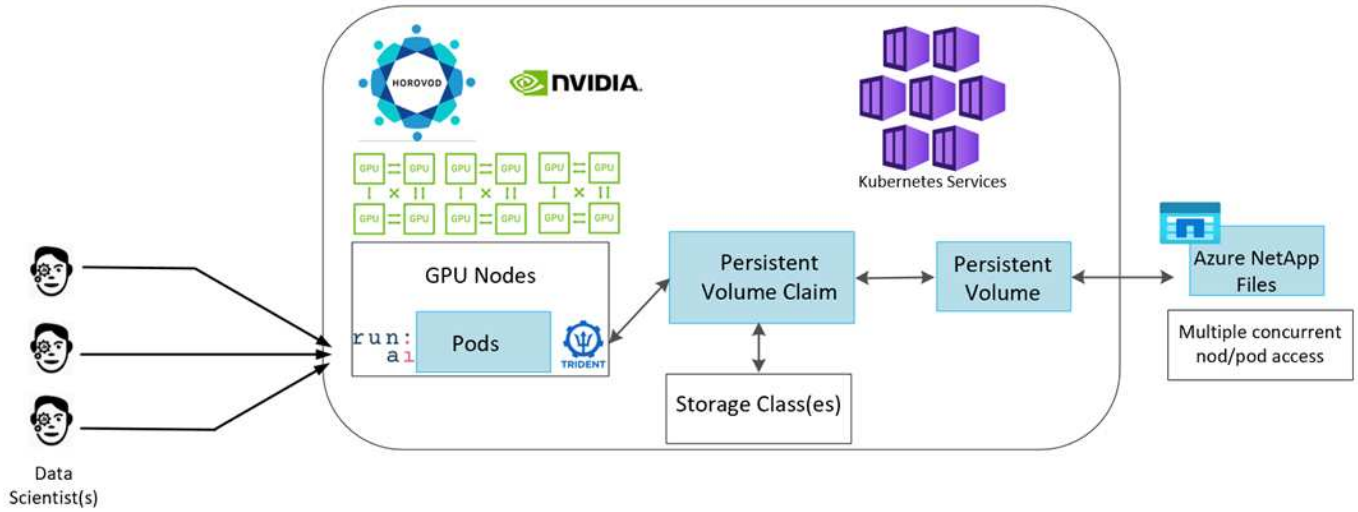
Lösungstechnologie

In diesem Abschnitt werden die technologischen Anforderungen für die Lane-Erkennung erläutert. Hierzu wird eine verteilte Trainingslösung in großem Umfang implementiert, die in der Azure Cloud ausgeführt wird. Die Abbildung unten bietet einen Überblick über die Lösungsarchitektur.

Folgende Elemente werden in dieser Lösung verwendet:

- Azure Kubernetes-Service (AKS)
- Azure Computing SKUs mit NVIDIA GPUs
- Azure NetApp Dateien
- AUSFÜHREN: KI
- NetApp Trident

Links zu allen hier genannten Elementen sind im aufgeführt "Weitere Informationen" Abschnitt.



Cloud-Ressourcen und -Serviceanforderungen

In der folgenden Tabelle werden die Hardwarekomponenten aufgeführt, die für die Implementierung der Lösung erforderlich sind. Je nach den Anforderungen des Kunden können die Cloud-Komponenten, die in einer beliebigen Implementierung dieser Lösung verwendet werden, variieren.

Cloud	Menge
AKS	Mindestens drei System-Nodes und drei GPU-Worker-Nodes
VM (Virtual Machine) SKU-System-Nodes	Drei Standard_DS2_v2
VM-SKU-GPU-Worker-Nodes	Drei Standard_NC6s_v3
Azure NetApp Dateien	4 TB Standard Tier

Softwareanforderungen

In der folgenden Tabelle werden die Softwarekomponenten aufgeführt, die für die Implementierung der Lösung erforderlich sind. Je nach den Anforderungen des Kunden können die in einer beliebigen Implementierung dieser Lösung verwendeten Softwarekomponenten abweichen.

Software	Version oder andere Informationen
AKS - Kubernetes-Version	1.18.14
AUSFÜHREN:AI-CLI	Version 2.2.25
RUN:AI Orchestration Kubernetes Operator Version	1.0.109
Horovod	0.21.2
NetApp Trident	20.01.1
Helm	3.0.0

Lane-Erkennung – verteiltes Training mit RUN:AI

Dieser Abschnitt enthält Details zum Einrichten der Plattform für die Durchführung von Distributed Training zur Lane-Erkennung mithilfe des RUN: AI Orchestrator. Wir sprechen über die Installation aller Lösungselemente und die Durchführung der verteilten Schulungen auf der besagten Plattform. DIE ML-Versionierung erfolgt über NetApp Snapshot, die mit RUN verbunden ist: KI-Experimente zur Erzielung von Daten und Modellreproduzierbarkeit. DIE ML-Versionierung spielt eine entscheidende Rolle bei Tracking-Modellen, bei der gemeinsamen Arbeit zwischen Teammitgliedern, bei der Reproduzierbarkeit der Ergebnisse, bei der Einführung neuer Modellversionen in die Produktion und bei der Datenerzeugung. NetApp ML Version Control (Snapshot) erfasst zeitpunktgenaue Versionen der Daten, geschulte Modelle und Protokolle für jedes Experiment. Die umfassende API-Unterstützung erleichtert die Integration IN DIE RUN: KI Plattform. Aufgrund des Trainingsstatus muss ein Ereignis nur ausgelöst werden. Man muss auch den Status des gesamten Experiments erfassen, ohne im Code oder in den Containern, die auf Kubernetes (K8s) laufen, etwas zu ändern.

Abschließend wird in diesem technischen Bericht die Performance-Bewertung auf mehreren GPU-fähigen Knoten in AKS durchgeführt.

Verteiltes Training für Lane-Erkennung Anwendungsfall mit dem TuSimple-Datensatz

In diesem technischen Bericht werden verteilte Schulungen auf dem TuSimple-Datensatz zur Fahrspurerkennung durchgeführt. Horovod wird im Trainingscode für das verteilte Daten-Training auf mehreren GPU-Nodes gleichzeitig im Kubernetes Cluster über AKS verwendet. Code wird als Container-Images für TuSimple-Daten-Download und -Verarbeitung verpackt. Verarbeitete Daten werden auf persistenten Volumes gespeichert, die durch das NetApp Trident Plug-in zugewiesen werden. Für das Training wird ein weiteres Container-Image erstellt und die Daten, die in persistenten Volumes gespeichert sind, werden beim Herunterladen der Daten verwendet.

Um den Daten- und Schulungsauftrag zu übermitteln, verwenden SIE RUN: KI zur Koordinierung von Ressourcenzuweisung und -Management. RUN: AI ermöglicht die Durchführung von MPI-Operationen (Message Passing Interface), die für Horovod benötigt werden. Mit diesem Layout können mehrere GPU-Knoten miteinander kommunizieren, um die Trainingsgewichte nach jedem Training-Mini-Batch zu aktualisieren. Zudem ermöglicht es das Monitoring des Trainings über die Benutzeroberfläche und die CLI, sodass der Fortschritt der Experimente einfach überwacht werden kann.

NetApp Snapshot ist in den Trainingscode integriert und erfasst den Status der Daten und das trainierte Modell für jedes Experiment. Damit können Sie die Version und den verwendeten Code sowie das damit verbundene trainierte Modell nachverfolgen.

AKS-Einrichtung und Installation

Zur Einrichtung und Installation des AKS-Clusters gehen Sie zu "Erstellen Sie einen AKS-Cluster". Führen Sie dann die folgenden Schritte aus:

1. Wählen Sie bei der Auswahl des Node-Typs (unabhängig davon, ob es sich um System- (CPU) oder „worker“-Nodes (GPU) handelt, Folgendes aus:

a. Fügen Sie den primären System-Node mit dem Namen hinzu `agentpool` Am `Standard_DS2_v2` Größe. Verwenden Sie die drei Standard-Nodes.

b. Fügen Sie den Node Worker hinzu `gpupool` Mit `the Standard_NC6s_v3` Pool-Größe. Verwenden Sie mindestens drei Nodes für GPU-Nodes.

Implementierung dauert 5 bis 10 Minuten.

2. Klicken Sie nach Abschluss der Bereitstellung auf mit Cluster verbinden. Um eine Verbindung mit dem neu erstellten AKS-Cluster herzustellen, installieren Sie das Kubernetes-Befehlszeilen-Tool aus Ihrer lokalen Umgebung (Laptop/PC). Besuchen Sie "Tools Installieren" So installieren Sie es nach Ihrem Betriebssystem.
3. "Installieren Sie die Azure CLI in Ihrer lokalen Umgebung".
4. Um über das Terminal auf den AKS-Cluster zuzugreifen, geben Sie zuerst ein `az login` Und geben Sie die Zugangsdaten ein.
5. Führen Sie die folgenden beiden Befehle aus:

```
az account set --subscription xxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx
aks get-credentials --resource-group resourcegroup --name aksclustername
```

6. Geben Sie diesen Befehl in die Azure CLI ein:

```
kubectl get nodes
```

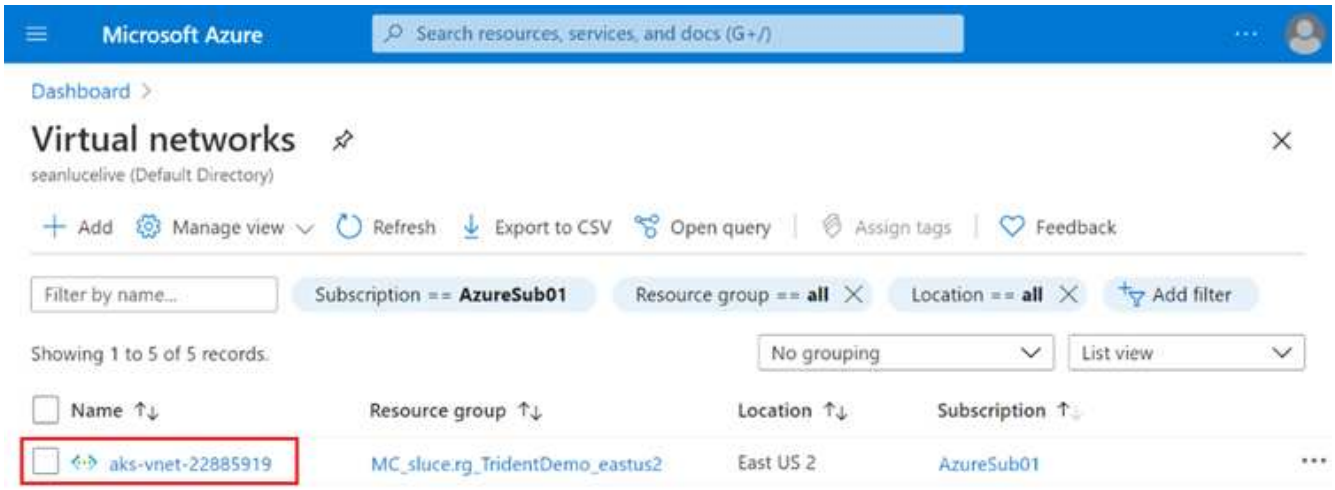
Wenn alle sechs Knoten wie hier dargestellt betriebsbereit sind, ist Ihr AKS-Cluster bereit und mit Ihrer lokalen Umgebung verbunden.

```
verronmartina@verron-mac-0 ~ % kubectl get nodes
NAME                                STATUS   ROLES   AGE   VERSION
aks-agentpool-34613062-vmss000000   Ready    agent   22m   v1.18.14
aks-agentpool-34613062-vmss000001   Ready    agent   22m   v1.18.14
aks-agentpool-34613062-vmss000002   Ready    agent   22m   v1.18.14
aks-gpupool-34613062-vmss000000     Ready    agent   20m   v1.18.14
aks-gpupool-34613062-vmss000001     Ready    agent   20m   v1.18.14
aks-gpupool-34613062-vmss000002     Ready    agent   20m   v1.18.14
verronmartina@verron-mac-0 ~ %
```

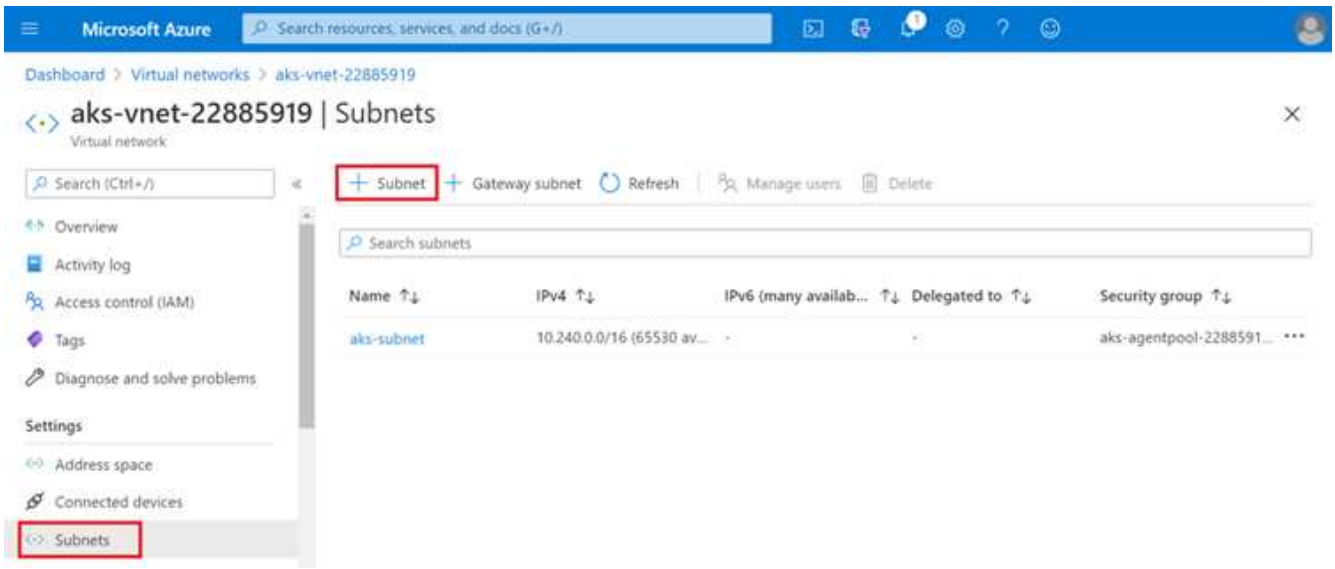
Erstellen Sie ein delegiertes Subnetz für Azure NetApp Files

Gehen Sie wie folgt vor, um ein delegiertes Subnetz für Azure NetApp Files zu erstellen:

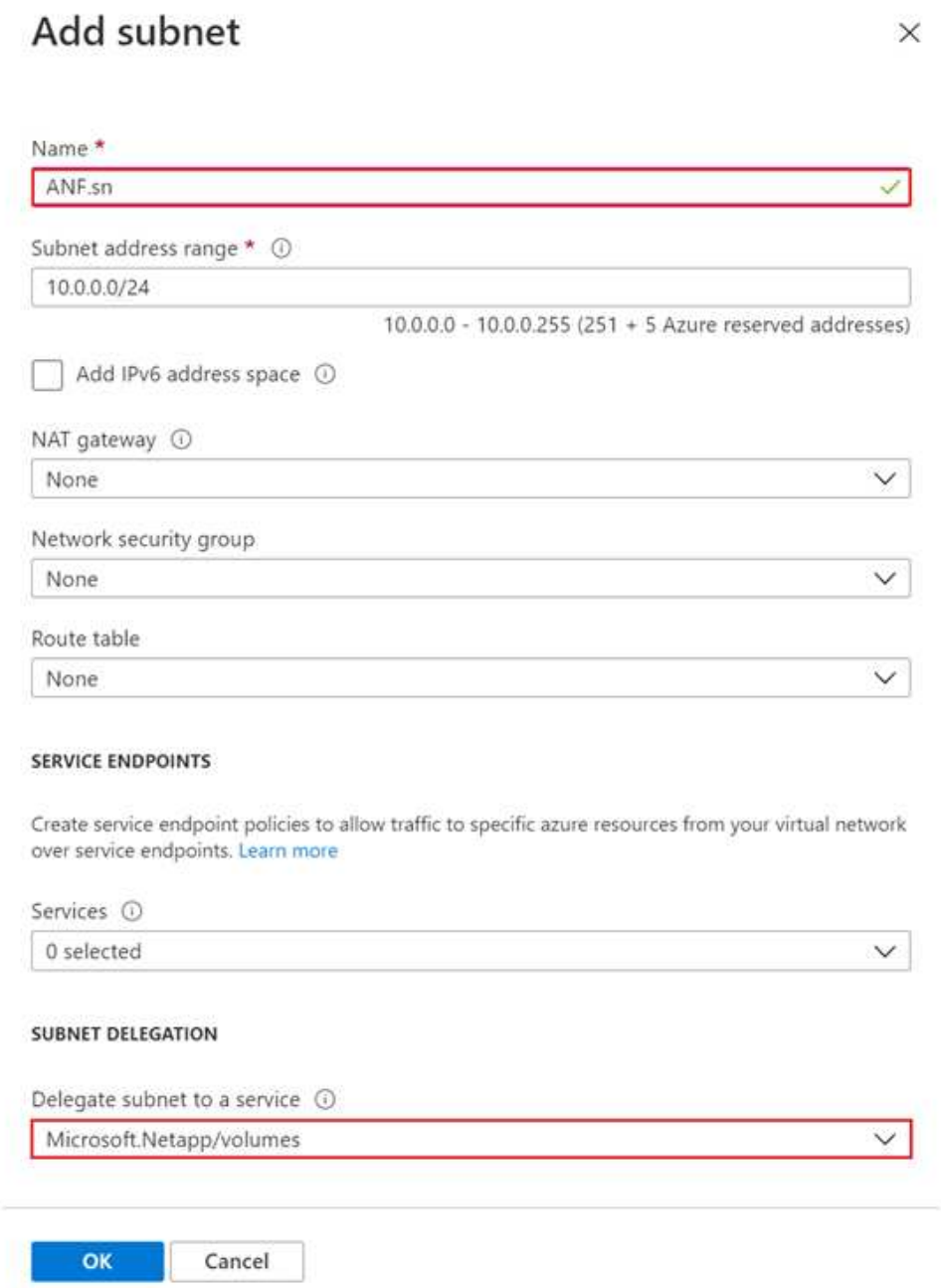
1. Navigieren Sie im Azure-Portal zu virtuellen Netzwerken. Suchen Sie Ihr neu erstelltes virtuelles Netzwerk. Es sollte ein Präfix wie Aaks-vnet haben, wie hier zu sehen. Klicken Sie auf den Namen des virtuellen Netzwerks.



2. Klicken Sie auf Subnetze, und wählen Sie in der oberen Symbolleiste +Subnetz aus.



3. Geben Sie dem Subnetz einen Namen an, z. B. `ANF.sn` Wählen Sie unter der Überschrift Subnet Delegation die Option Microsoft.NetApp/volumes aus. Ändern Sie nichts anderes. Klicken Sie auf OK.



Azure NetApp Files Volumes werden dem Applikations-Cluster zugewiesen und als persistente Volume-Forderungen (PVCs) in Kubernetes genutzt. Diese Zuweisung wiederum bietet uns die Flexibilität, Volumes verschiedenen Services zuzuordnen, sei es bei Jupyter Notebooks, serverlosen Funktionen usw.

Benutzer von Services können Storage auf unterschiedliche Weise von der Plattform aus nutzen. Azure NetApp Files bietet folgende Hauptvorteile:

- Bietet Benutzern die Möglichkeit, Snapshots zu verwenden.
- Ermöglicht Benutzern die Speicherung großer Datenmengen auf Azure NetApp Files Volumes.
- Beziehen Sie sich die Performance-Vorteile von Azure NetApp Files Volumes durch Ausführung ihrer Modelle auf große Dateimengen.

Einrichtung von Azure NetApp Files

Um die Einrichtung von Azure NetApp Files abzuschließen, müssen Sie sie zuerst wie in beschrieben konfigurieren "QuickStart: Azure NetApp Files einrichten und ein NFS-Volume erstellen".

Sie können jedoch die Schritte zur Erstellung eines NFS Volumes für Azure NetApp Files weglassen, wie Sie Volumes über Trident erstellen. Bevor Sie fortfahren, sollten Sie Folgendes beachten:

1. "Registriert für Azure NetApp Files und NetApp Resource Provider (über den Azure Cloud Shell)".
2. "In Azure NetApp Files wurde ein Konto erstellt".
3. "Richten Sie einen Kapazitäts-Pool ein" (Mindestens 4 tib Standard oder Premium, je nach Ihren Anforderungen)

Peering von virtuellem AKS-Netzwerk und virtuellem Azure NetApp Files-Netzwerk

Führen Sie als Nächstes die folgenden Schritte aus, um das virtuelle AKS-Netzwerk (vnet) mit dem Azure NetApp Files vnet in Verbindung zu setzen:

1. Geben Sie in das Suchfeld oben im Azure-Portal virtuelle Netzwerke ein.
2. Klicken Sie auf vnet aks- vnet-Name, und geben Sie dann Peerings in das Suchfeld ein.
3. Klicken Sie auf + Hinzufügen, und geben Sie die Informationen in der folgenden Tabelle ein:

Feld	Wert oder Beschreibung
Linkname des Peering-Links	aks-vnet-Name_to_anf
SubskriptionID	Abonnement des Azure NetApp Files vnet, zu dem Sie spähen
Vnet Peering-Partner	Azure NetApp Files vnet

> Lassen Sie alle nicht-Sternchen-Abschnitte standardmäßig unverändert

4. Klicken Sie AUF HINZUFÜGEN oder OK, um das Peering zum virtuellen Netzwerk hinzuzufügen.

Weitere Informationen finden Sie unter "Virtuelles Netzwerk-Peering erstellen, ändern oder löschen".

Trident

Trident ist ein Open-Source-Projekt von NetApp für persistenten Storage für Applikations-Container. Trident wird als externer Controller für die bereitstellung implementiert, der selbst als Pod ausgeführt wird. Mit ihm werden Volumes überwacht und der Bereitstellungsprozess vollständig automatisiert.

NetApp Trident ermöglicht eine reibungslose Integration in K8s, indem persistente Volumes zum Speichern von Trainingsdatensätzen und trainierten Modellen erstellt und angehängt werden. So können Data Scientists und Data Engineers K8s einfacher verwenden – ohne die manuelle Speicherung und das manuelle Management von Datensätzen. Mit Trident müssen Data Scientists zudem keine Erfahrung mehr mit dem Management neuer Datenplattformen machen, da die Datenmanagement-Aufgaben durch die Integration der logischen API integriert werden.

Installation Von Trident

So installieren Sie die Trident Software:

1. "Zuerst Helm einbauen".
2. Laden Sie das Trident 21.01.1-Installationsprogramm herunter und extrahieren Sie es.

```
wget
https://github.com/NetApp/trident/releases/download/v21.01.1/trident-
installer-21.01.1.tar.gz
tar -xf trident-installer-21.01.1.tar.gz
```

3. Ändern Sie das Verzeichnis in `trident-installer`.

```
cd trident-installer
```

4. Kopieren `tridentctl` In ein Verzeichnis im System `$PATH`.

```
cp ./tridentctl /usr/local/bin
```

5. Installation von Trident auf K8s Cluster mit Helm:
   a. Verzeichnis in Steuerverzeichnis ändern.

   ```
   cd helm
   ```

   b. Installation Von Trident:

   ```
   helm install trident trident-operator-21.01.1.tgz --namespace trident
   --create-namespace
   ```

   c. Überprüfen Sie den Status von Trident Pods die übliche K8s Art und Weise:

   ```
   kubectl -n trident get pods
   ```

   d. Wenn alle Pods in Betrieb sind, ist Trident installiert und Sie können gut aufgestellt werden.

Richten Sie das Azure NetApp Files Back-End und die Storage-Klasse ein

Gehen Sie wie folgt vor, um das Azure NetApp Files Back-End und die Storage-Klasse einzurichten:

1. Wechseln Sie zurück zum Home-Verzeichnis.

```
cd ~
```

2. Klonen Sie die "Projekt-Repository" `lane-detection-SCNN-horovod`.
3. Wechseln Sie zum `trident-config` Verzeichnis.

```
cd ./lane-detection-SCNN-horovod/trident-config
```

4. Erstellung eines Azure-Serviceprinzips (das Service-Prinzip besteht darin, wie Trident mit Azure kommuniziert, um auf Ihre Azure NetApp Files-Ressourcen zuzugreifen).

```
az ad sp create-for-rbac --name
```

Die Ausgabe sollte wie im folgenden Beispiel aussehen:

```
{
  "appId": "xxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
   "displayName": "netapptrident",
    "name": "http://netapptrident",
    "password": "xxxxxxxxxxxxxx.xxxxxxxxxxxxx",
    "tenant": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxx"
 }
```

5. Erstellen Sie das Trident `backend json` Datei:
6. Füllen Sie mithilfe Ihres bevorzugten Texteditors die folgenden Felder aus der Tabelle unten im aus `anf-backend.json` Datei:

Feld	Wert
SubskriptionID	Ihre Azure-Abonnement-ID
TenantID	Ihre Azure Mandanten-ID (aus der Ausgabe von az ad sp im vorherigen Schritt)
Client-ID	Ihre appID (aus der Ausgabe von az ad sp im vorherigen Schritt)
ClientSecret	Ihr Kennwort (aus der Ausgabe von az ad sp im vorherigen Schritt)

Die Datei sollte wie das folgende Beispiel aussehen:

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "fakec765-4774-fake-ae98-a721add4fake",
    "tenantID": "fakef836-edc1-fake-bff9-b2d865eefake",
    "clientID": "fake0f63-bf8e-fake-8076-8de91e57fake",
    "clientSecret": "SECRET",
    "location": "westeurope",
    "serviceLevel": "Standard",
    "virtualNetwork": "anf-vnet",
    "subnet": "default",
    "nfsMountOptions": "vers=3,proto=tcp",
    "limitVolumeSize": "500Gi",
    "defaults": {
    "exportRule": "0.0.0.0/0",
    "size": "200Gi"
}
```

7. Weisen Sie Trident an, das Azure NetApp Files-Back-End im zu erstellen `trident` Namespace verwenden `anf-backend.json` Die Konfigurationsdatei ist wie folgt:

```
tridentctl create backend -f anf-backend.json -n trident
```

8. Speicherklasse erstellen:

   a. K8 Benutzer stellen Volumes mithilfe von PVCs bereit, die eine Storage-Klasse nach Namen angeben. Weisen Sie K8s an, eine Speicherklasse zu erstellen `azurenetappfiles` Diese Referenz wird auf das im vorherigen Schritt erstellte Azure NetApp Files Back-End verweisen:

   ```
   kubectl create -f anf-storage-class.yaml
   ```

   b. Überprüfen Sie, ob Storage-Klassen mit folgendem Befehl erstellt werden:

   ```
   kubectl get sc azurenetappfiles
   ```

   Die Ausgabe sollte wie im folgenden Beispiel aussehen:

```
NAME               PROVISIONER            RECLAIMPOLICY   VOLUMEBINDINGMODE   ALLOWVOLUMEEXPANSION   AGE
azurenetappfiles   csi.trident.netapp.io  Delete          Immediate           false                  98s
```

Bereitstellen und Einrichten von Volume Snapshot-Komponenten auf AKS

Wenn Ihr Cluster nicht mit den korrekten Volume-Snapshot-Komponenten vorinstalliert wird, können Sie diese Komponenten manuell installieren, indem Sie die folgenden Schritte ausführen:

AKS 1.18.14 verfügt nicht über einen vorinstallierten Snapshot-Controller.

1. Installieren Sie Snapshot Beta-CRDs unter Verwendung der folgenden Befehle:

```
kubectl create -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
3.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshotclasses.yaml
kubectl create -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
3.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshotcontents.yam
l
kubectl create -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
3.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshots.yaml
```

2. Installieren Sie Snapshot Controller mithilfe der folgenden Dokumente von GitHub:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-3.0/deploy/kubernetes/snapshot-
controller/rbac-snapshot-controller.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-3.0/deploy/kubernetes/snapshot-
controller/setup-snapshot-controller.yaml
```

3. K8s einrichten `volumesnapshotclass`: Vor der Erstellung eines Volume-Snapshot, a "Volume Snapshot-Klasse" Muss eingerichtet werden. Erstellen Sie einen Volume-Snapshot für Azure NetApp Files, und erstellen Sie mit dieser Technologie eine ML-Versionierung. Erstellen `volumesnapshotclass netapp-csi-snapclass` Und stellen Sie ihn als Standard `volumesnapshotclass `wie folgt ein:

```
kubectl create -f netapp-volume-snapshot-class.yaml
```

Die Ausgabe sollte wie im folgenden Beispiel aussehen:

```
volumesnapshotclass.snapshot.storage.k8s.io/netapp-csi-snapclass created
```

4. Überprüfen Sie, ob die Klasse der Volume Snapshot Kopien mithilfe des folgenden Befehls erstellt wurde:

```
kubectl get volumesnapshotclass
```

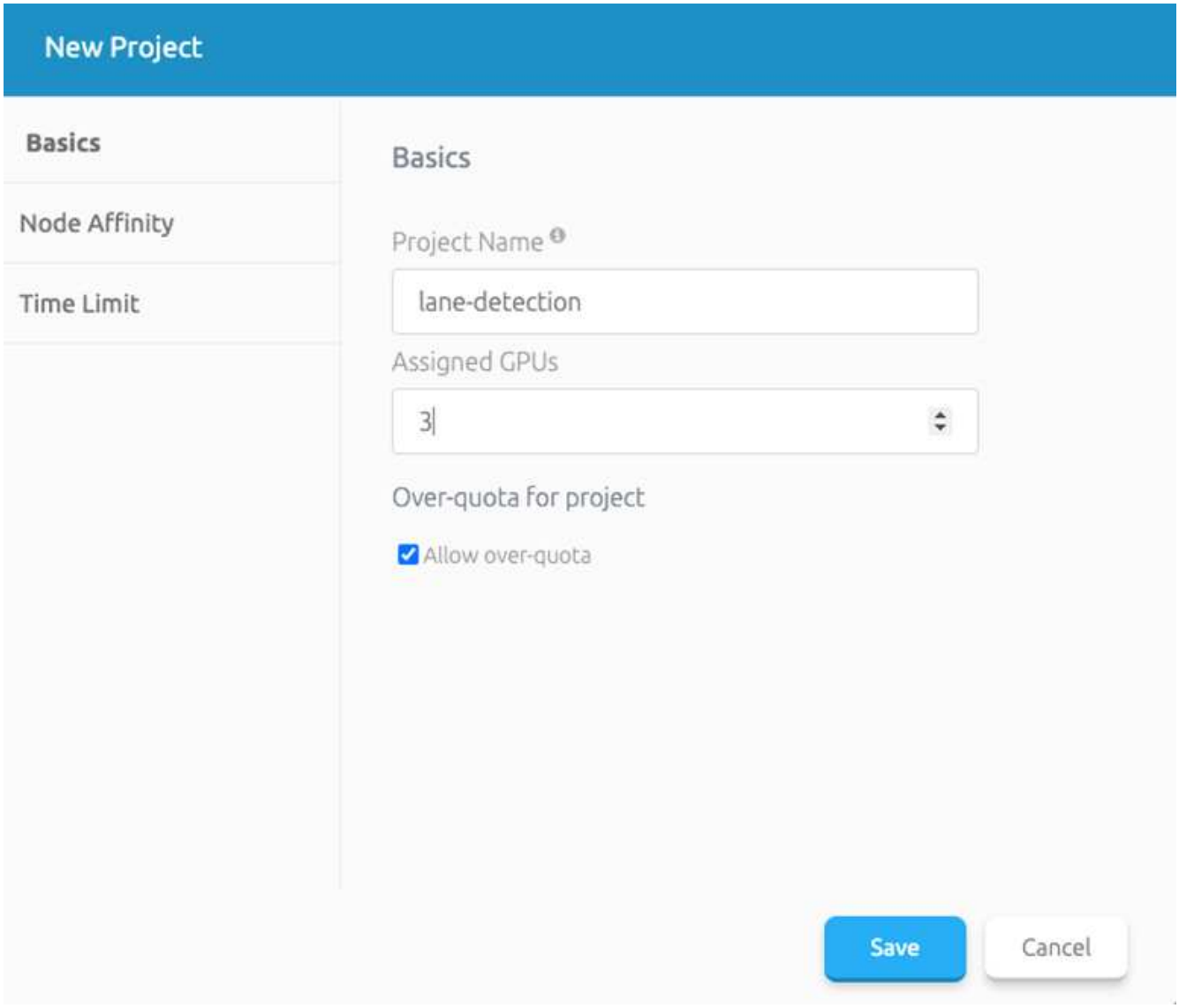
Die Ausgabe sollte wie im folgenden Beispiel aussehen:

```
NAME                    DRIVER                  DELETIONPOLICY   AGE
netapp-csi-snapclass    csi.trident.netapp.io   Delete           63s
```

RUN:AI Installation

So installieren SIE RUN:AI:

1. "Installieren SIE RUN:AI Cluster auf AKS".
2. Gehen Sie zu app.runai.ai, klicken Sie auf Neues Projekt erstellen und benennen Sie es Lane-Detection. Es wird einen Namespace auf einem K8s-Cluster erstellen, der mit beginnt `runai-` Gefolgt vom Projektnamen. In diesem Fall wäre der erstellte Namespace Runai-Lane-Erkennung.



3. "INSTALLIEREN SIE RUN:AI CLI".
4. Stellen Sie auf Ihrem Terminal standardmäßig die Lane-Detection ein: AI-Projekt mit folgendem Befehl:

```
`runai config project lane-detection`
```

Die Ausgabe sollte wie im folgenden Beispiel aussehen:

```
Project lane-detection has been set as default project
```

5. ClusterRole und ClusterRoleBinding für den Projekt-Namespace erstellen (z. B. `lane-detection`) Also das Standard-Servicekonto, das zu gehört `runai-lane-detection` Namespace hat die Berechtigung zum Ausführen `volumesnapshot` Operationen während der Jobausführung:

   a. Listen Sie Namespaces auf, um das zu überprüfen `runai-lane-detection` Existiert durch Verwendung dieses Befehls:

   ```
   kubectl get namespaces
   ```

   Die Ausgabe sollte wie im folgenden Beispiel erscheinen:

   ```
   NAME                  STATUS   AGE
   default               Active   130m
   kube-node-lease       Active   130m
   kube-public           Active   130m
   kube-system           Active   130m
   runai                 Active   4m44s
   runai-lane-detection  Active   13s
   trident               Active   102m
   ```

6. ClusterCole erstellen `netappsnapshot` Und ClusterRoleBending `netappsnapshot` Verwenden der folgenden Befehle:

   ```
   `kubectl create -f runai-project-snap-role.yaml`
   `kubectl create -f runai-project-snap-role-binding.yaml`
   ```

Den TuSimple-Datensatz als RUN:AI-Job herunterladen und verarbeiten

Der Prozess zum Herunterladen und Verarbeiten des TuSimple-Datensatzes als RUN: AI-Job ist optional. Sie umfasst folgende Schritte:

1. Erstellen und Drücken Sie das Docker-Bild, oder lassen Sie diesen Schritt aus, wenn Sie ein vorhandenes Docker-Bild verwenden möchten (z. B. `muneer7589/download-tusimple:1.0`)

   a. Zum Home-Verzeichnis wechseln:

   ```
   cd ~
   ```

   b. Gehen Sie zum Datenverzeichnis des Projekts `lane-detection-SCNN-horovod`:

   ```
   cd ./lane-detection-SCNN-horovod/data
   ```

c. Ändern `build_image.sh` Shell-Skript und ändern Docker-Repository zu Ihrem. Beispiel: Ersetzen `muneer7589` Mit dem Namen des Docker-Repositorys. Sie können auch den Namen und DAS TAG des Docker-Images ändern (z. B. `download-tusimple` Und `1.0`):

```
#!/bin/bash
#
# A simple script to build the Docker image.
#
# $ build_image.sh
set -ex

IMAGE=muneer7589/download-tusimple
TAG=1.0

# Build image
echo "Building image: "$IMAGE
docker build . -f Dockerfile \
 --tag "${IMAGE}:${TAG}"
echo "Finished building image: "$IMAGE

# Push image
echo "Pushing image: "$IMAGE
docker push "${IMAGE}:${TAG}"
echo "Finished pushing image: "$IMAGE
```

d. Führen Sie das Skript aus, um das Docker-Image zu erstellen und es mithilfe folgender Befehle in das Docker-Repository zu verschieben:

```
chmod +x build_image.sh
./build_image.sh
```

2. Senden Sie DEN RUN: AI Job zum Herunterladen, Extrahieren, Vorverarbeiten und Speichern der TuSimple Lane Detection Dataset in a `pvc`, Das von NetApp Trident dynamisch erstellt wird:

a. Übermitteln Sie den JOB „RUN: AI“ mithilfe der folgenden Befehle:

```
runai submit
--name download-tusimple-data
--pvc azurenetappfiles:100Gi:/mnt
--image muneer7589/download-tusimple:1.0
```

b. Geben Sie die Informationen aus der Tabelle unten ein, um den JOB RUN:AI einzureichen:

Feld	Wert oder Beschreibung
-Name	Name des Jobs
-pvc	PVC des Formats [StorageClassName]:Größe:ContainerMountPath in der oben genannten Jobeinreichung erstellen Sie ein PVC-basiertes On-Demand mit Trident mit Speicherklasse azurenetappfiles. Persistente Volumen Kapazität hier ist 100Gi und es ist an Pfad /mnt montiert.
-Image	Das Docker-Image sollte beim Erstellen des Containers für diesen Job verwendet werden

Die Ausgabe sollte wie im folgenden Beispiel aussehen:

```
The job 'download-tusimple-data' has been submitted successfully
You can run `runai describe job download-tusimple-data -p lane-detection` to check the job status
```

c. Listen Sie die eingereichten RUN:AI-Jobs auf.

```
runai list jobs
```

```
Showing jobs for project lane-detection
NAME                    STATUS            AGE  NODE                          IMAGE                          TYPE  PROJECT        USER          GPUs Allocated (Requested)
PODs Running (Pending)  SERVICE URL(S)
download-tusimple-data  ContainerCreating  1m   aks-agentpool-34613062-vmss00000a  muneer7589/download-tusimple:1.0  Train  lane-detection  verronmartina  0 (0)
1 (0)
```

d. Überprüfen Sie die eingereichten Jobprotokolle.

```
runai logs download-tusimple-data -t 10
```

```
751150K .......... .......... .......... .......... ..........  6% 16.2M 20m37s
751200K .......... .......... .......... .......... ..........  6% 11.1M 20m37s
751250K .......... .......... .......... .......... ..........  6% 12.5M 20m36s
751300K .......... .......... .......... .......... ..........  6% 11.3M 20m36s
751350K .......... .......... .......... .......... ..........  6% 15.2M 20m36s
751400K .......... .......... .......... .......... ..........  6% 10.5M 20m36s
751450K .......... .......... .......... .......... ..........  6% 15.2M 20m36s
751500K .......... .......... .......... .......... ..........  6% 14.1M 20m36s
751550K .......... .......... .......... .......... ..........  6% 24.3M 20m36s
751600K .......... .......... .......... .......... ..........  6% 26.3M 20m36s
```

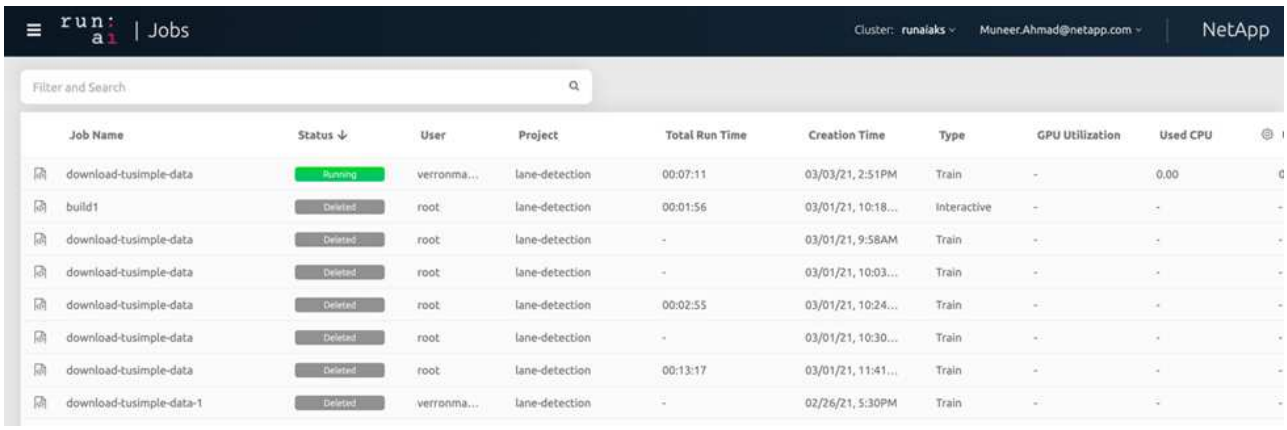
e. Listen Sie die auf `pvc` Erstellt. Verwenden Sie diese Option `pvc` Befehl für Training im nächsten Schritt.

```
kubectl get pvc | grep download-tusimple-data
```

Die Ausgabe sollte wie im folgenden Beispiel aussehen:

```
pvc-download-tusimple-data-0   Bound    pvc-bb03b74d-2c17-40c4-a445-79f3de8d16d5   100Gi      RWO            azurenetappfiles   4m47s
```

a. Prüfen Sie DEN Job ausgeführt: KI-UI (oder `app.run.ai`).



Führen Sie mithilfe von Horovod eine Schulung zur Erkennung verteilter Fahrspuren durch

Das Training zur Distributed Lane Detection mit Horovod ist ein optionaler Prozess. Hier sind jedoch die Schritte zu beachten:

1. Erstellen und Drücken Sie das Docker-Bild, oder überspringen Sie diesen Schritt, wenn Sie das vorhandene Docker-Bild verwenden möchten (z. B. `muneer7589/dist-lane-detection:3.1)`:

   a. Wechseln Sie zum Home Directory.

   ```
   cd ~
   ```

   b. Rufen Sie das Projektverzeichnis auf `lane-detection-SCNN-horovod.`

   ```
   cd ./lane-detection-SCNN-horovod
   ```

   c. Ändern Sie die `build_image.sh` Shell-Skript und ändern Docker-Repository zu Ihrem (z. B. Ersetzen `muneer7589` Mit dem Namen des Docker-Repository). Sie können auch den Namen und DAS TAG des Dockers ändern (`dist-lane-detection` Und `3.1, for example).`

```
#!/bin/bash
#
# A simple script to build the distributed Docker image.
#
# $ build_image.sh
set -ex

IMAGE=muneer7589/dist-lane-detection
TAG=3.0

# Build image
echo "Building image: "$IMAGE
docker build . -f Dockerfile \
 --tag "${IMAGE}:${TAG}"
echo "Finished building image: "$IMAGE

# Push image
echo "Pushing image: "$IMAGE
docker push "${IMAGE}:${TAG}"
echo "Finished pushing image: "$IMAGE
```

d. Führen Sie das Skript aus, um das Docker-Image zu erstellen, und drücken Sie zum Docker-Repository.

```
chmod +x build_image.sh
./build_image.sh
```

2. RUN: AI Job zur Durchführung von Distributed Training (MPI):

a. Verwendung von Run: AI zur automatischen Erstellung von PVC im vorherigen Schritt (zum Herunterladen von Daten) ermöglicht nur RWO-Zugriff, sodass nicht mehrere Pods oder Knoten zum verteilten Training auf dasselbe PVC zugreifen können. Aktualisieren Sie den Zugriffsmodus auf ReadWriteManche und verwenden Sie dazu den Kubernetes-Patch.

b. Erhalten Sie zunächst den Volume-Namen des PVC durch Ausführen des folgenden Befehls:

```
kubectl get pvc | grep download-tusimple-data
```

```
root@ai-w-gpu-2:/mnt/ai_data/anf_runai/lane-detection-SCNN-horovod# kubectl get pvc | grep download-tusimple-data
pvc-download-tusimple-data-0   Bound    pvc-bb03b74d-2c17-40c4-a445-79f3de8d16d5   100Gi      RWX            azurenetappfiles   2d4h
```

c. Patchen des Volume und Aktualisieren des Zugriffsmodus auf ReadWriteManche (ersetzen Sie den Datenträgernamen durch Ihren im folgenden Befehl):

```
kubectl patch pv pvc-bb03b74d-2c17-40c4-a445-79f3de8d16d5 -p
'{"spec":{"accessModes":["ReadWriteMany"]}}'
```

d. Senden Sie DEN JOB RUN: AI MPI zur Ausführung des Jobs für verteilte Schulungen` mithilfe von Informationen aus der folgenden Tabelle:

```
runai submit-mpi
--name dist-lane-detection-training
--large-shm
--processes=3
--gpu 1
--pvc pvc-download-tusimple-data-0:/mnt
--image muneer7589/dist-lane-detection:3.1
-e USE_WORKERS="true"
-e NUM_WORKERS=4
-e BATCH_SIZE=33
-e USE_VAL="false"
-e VAL_BATCH_SIZE=99
-e ENABLE_SNAPSHOT="true"
-e PVC_NAME="pvc-download-tusimple-data-0"
```

Feld	Wert oder Beschreibung
Name	Name des Distributed Training Job
Großer shm	Ein großes /dev/shm-Gerät mounten Es ist ein auf RAM montiertes Shared-Dateisystem und bietet genügend gemeinsamen Speicher für mehrere CPU-Mitarbeiter, um Batches in CPU-RAM zu verarbeiten und zu laden.
Prozessen	Anzahl der verteilten Trainingsprozesse
gpu	Anzahl der GPUs/Prozesse, die für die Aufgabe in diesem Job zugewiesen werden sollen, es gibt drei GPU-Worker-Prozesse (--processes=3), die jeweils über eine einzelne GPU (--gpu 1) zugewiesen sind.
pvc	Verwenden Sie das vorhandene persistente Volume (pvc-download-tusimple-Data-0), das von einem vorherigen Job erstellt wurde (download-tusimple-Data), und es wird in Pfad /mnt bereitgestellt
Bild	Das Docker-Image sollte beim Erstellen des Containers für diesen Job verwendet werden
Definieren Sie Umgebungsvariablen, die im Container festgelegt werden sollen	
VERWENDEN VON MITARBEITERN	Wenn Sie das Argument auf true setzen, wird das Laden von mehreren Prozessdaten aktiviert
NUM_WORKERS	Anzahl der Data Loader Worker Prozesse
BATCH_SIZE	Batch-Größe für Training
NUTZUNG_VAL	Wenn Sie das Argument auf true setzen, kann die Validierung aktiviert werden
VAL_BATCH_SIZE	Batch-Größe der Validierung

Feld	Wert oder Beschreibung
AKTIVIEREN_SNAPSHOT	Wenn Sie das Argument auf true setzen, können Sie Daten und trainierte Modellschnappschüsse für ML-Versionierung erstellen
PVC-NAME	Name des pvc, von dem ein Snapshot erstellt werden soll. In der oben genannten Jobsendung erstellen Sie eine Momentaufnahme von pvc-Download-Tusimple-Data-0, bestehend aus Datensatz und trainierten Modellen

Die Ausgabe sollte wie im folgenden Beispiel aussehen:

```
The job 'dist-lane-detection-training' has been submitted successfully
You can run `runai describe job dist-lane-detection-training -p lane-detection` to check the job status
```

e. Geben Sie den eingereichten Job an.

```
runai list jobs
```

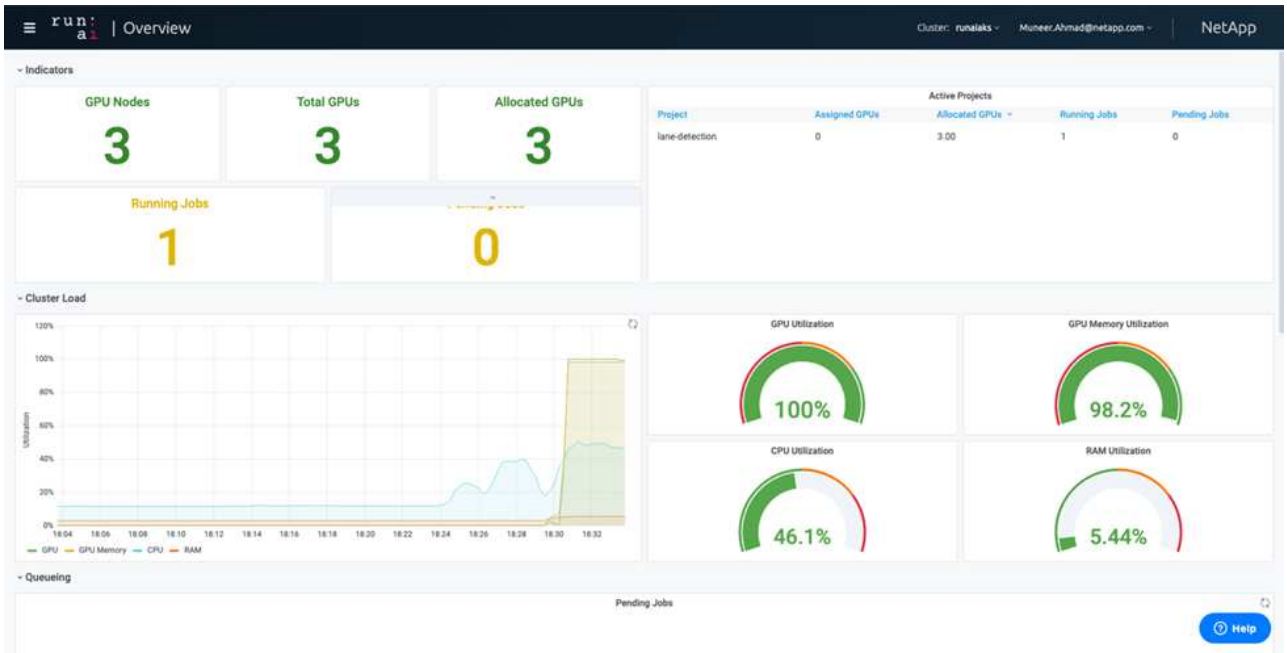
```
NAME                          STATUS     AGE  NODE        IMAGE                              TYPE   PROJECT         USER           GPUs Allocated (Requested)  PODs
 SERVICE URL(S)
download-tusimple-data        Succeeded  1d                muneer7589/download-tusimple:1.0   Train  lane-detection  verronmartina  - (0)                       0 (0)

dist-lane-detection-training  Init:0/1   2m   <multiple>  muneer7589/dist-lane-detection:3.1 Train  lane-detection  root           3 (3)                       4 (0)
```

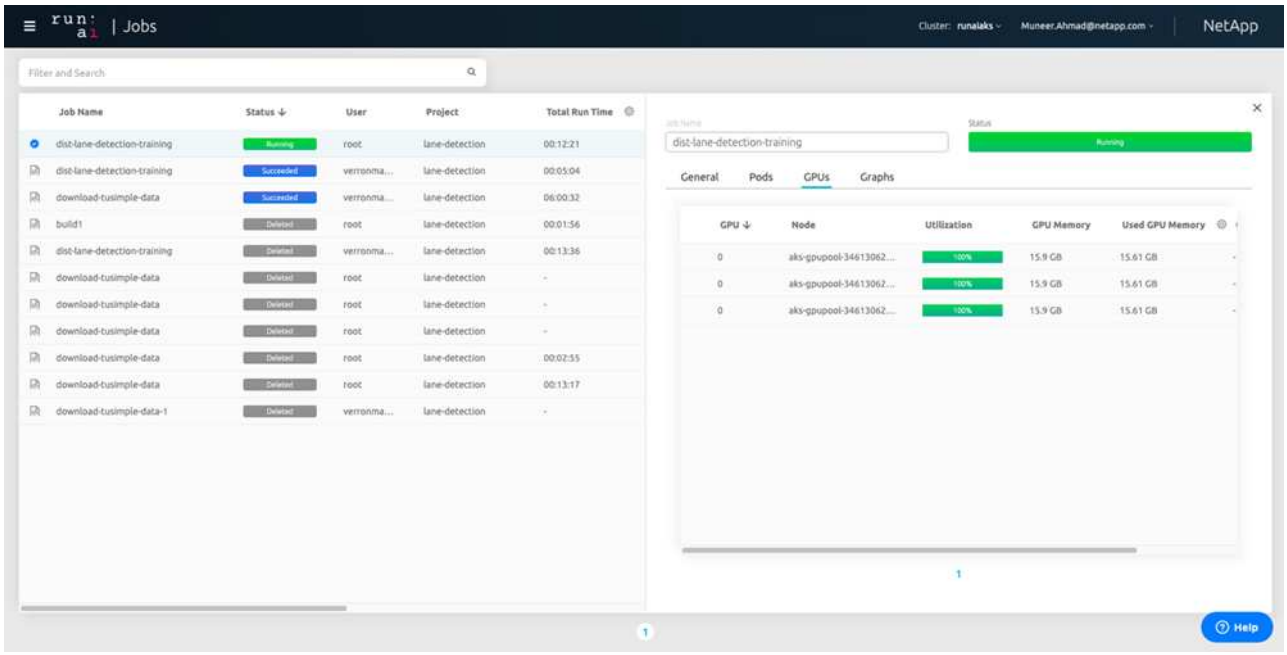
f. Eingereichte Jobprotokolle:

```
runai logs dist-lane-detection-training
```

```
root@ai-w-gpu-2:~/runai# runai logs dist-lane-detection-training
Running with 3 workers
2021-03-04 17:29:23.158449: I tensorflow/stream_executor/platform/default/dso_loader.cc:48] Successfully opened dynamic library libcudart.so.10.1
+ POD_NAME=dist-lane-detection-training-worker-0
+ [ d = - ]
+ shift
+ /opt/kube/kubectl cp /opt/kube/hosts dist-lane-detection-training-worker-0:/etc/hosts_of_nodes
+ POD_NAME=dist-lane-detection-training-worker-2
+ [ d = - ]
+ shift
+ /opt/kube/kubectl cp /opt/kube/hosts dist-lane-detection-training-worker-2:/etc/hosts_of_nodes
+ POD_NAME=dist-lane-detection-training-worker-1
```

g. Prüfen Sie den Trainingsjob IN AUSFÜHRUNG: AI GUI (oder app.runai.ai): RUN: AI Dashboard, wie in den Abbildungen unten zu sehen. Die erste Abbildung zeigt drei GPUs, die für den verteilten Trainingsjob auf drei Knoten auf AKS verteilt sind, und den zweiten DURCHLAUF:KI-Jobs:





h. Prüfen Sie nach Abschluss des Trainings die NetApp Snapshot Kopie, die erstellt wurde und mit RUN: KI-Job verknüpft ist.

```
runai logs dist-lane-detection-training --tail 1
```

```
[1,0]<stdout>:Snapshot snap-pvc-download-tusimple-data-0-dist-lane-detection-training-launcher-2021-03-05-16-23-42 created in namespace runai-lane-detection
```

```
kubectl get volumesnapshots | grep download-tusimple-data-0
```

Wiederherstellung von Daten aus der NetApp Snapshot Kopie

Um Daten aus der NetApp Snapshot Kopie wiederherzustellen, gehen Sie wie folgt vor:

1. Wechseln Sie zum Home Directory.

```
cd ~
```

2. Rufen Sie das Projektverzeichnis auf `lane-detection-SCNN-horovod`.

```
cd ./lane-detection-SCNN-horovod
```

3. Ändern `restore-snaphot-pvc.yaml` Und Aktualisierung `dataSource name` Feld zur Snapshot Kopie, aus der Sie Daten wiederherstellen möchten. Sie können auch den PVC-Namen ändern, in dem die Daten wiederhergestellt werden, in diesem Beispiel ist `restored-tusimple`.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: restored-tusimple
spec:
  storageClassName: azurenetappfiles
  dataSource:
    name: snap-pvc-download-tusimple-data-0-dist-lane-detection-training-launcher-2021-03-05-16-23-42
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 100Gi
```

4. Erstellen Sie mithilfe von ein neues PVC `restore-snapshot-pvc.yaml`.

```
kubectl create -f restore-snapshot-pvc.yaml
```

Die Ausgabe sollte wie im folgenden Beispiel aussehen:

```
persistentvolumeclaim/restored-tusimple created
```

5. Wenn Sie die gerade wiederhergestellten Daten für die Schulung verwenden möchten, bleibt die Bewerbung gleich wie zuvor; ersetzen Sie nur die `PVC_NAME` Mit dem wiederhergestellten `PVC_NAME` Beim Einreichen des Schulungsjobs, wie in den folgenden Befehlen zu sehen:

```
runai submit-mpi
--name dist-lane-detection-training
--large-shm
--processes=3
--gpu 1
--pvc restored-tusimple:/mnt
--image muneer7589/dist-lane-detection:3.1
-e USE_WORKERS="true"
-e NUM_WORKERS=4
-e BATCH_SIZE=33
-e USE_VAL="false"
-e VAL_BATCH_SIZE=99
-e ENABLE_SNAPSHOT="true"
-e PVC_NAME="restored-tusimple"
```

Performance-Bewertung

Um die lineare Skalierbarkeit der Lösung zu zeigen, wurden Performance-Tests für zwei Szenarien durchgeführt: Eine GPU und drei GPUs. GPU-Zuweisung, GPU- und Arbeitsspeicherauslastung sowie verschiedene Single- und drei-Node-Metriken wurden während des Trainings im TuSimple Lane-Erkennungsdatensatz erfasst. Die Datenmenge wird um das fünf- fache erhöht, nur um die Ressourcenauslastung während der Trainingsprozesse zu analysieren.

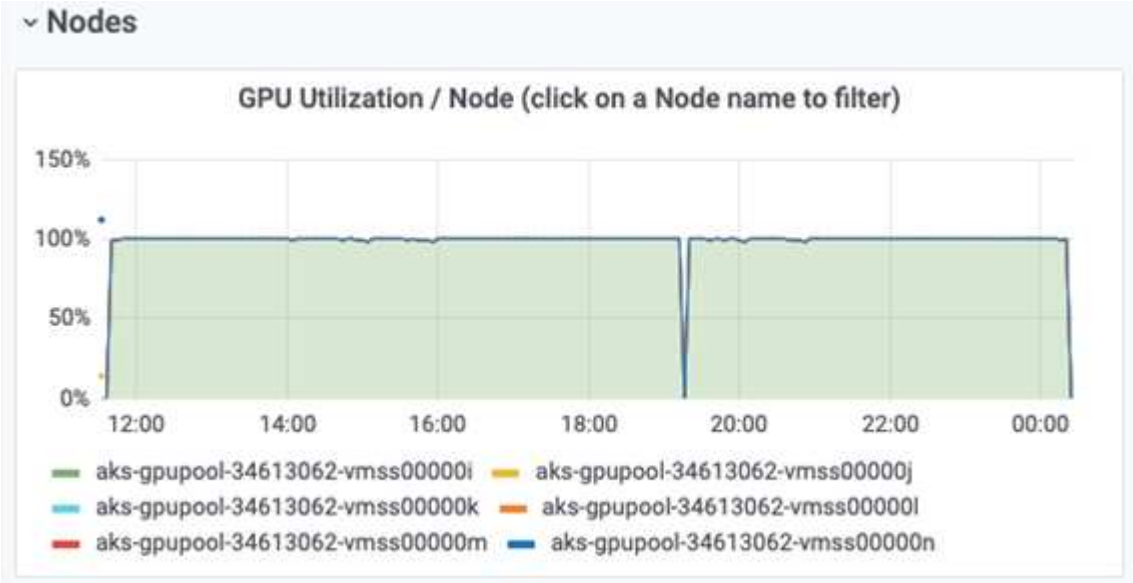
Die Lösung ermöglicht es Kunden, mit einem kleinen Datensatz und einigen GPUs zu beginnen. Wenn die Datenmenge und der Bedarf der GPUs steigen, können Kunden die Terabyte im Standard-Tier dynamisch horizontal skalieren und schnell auf die Premium-Tier skalieren. So wird der vierfache Durchsatz pro Terabyte erzielt, ohne Daten zu verschieben. Dieser Prozess wird im Abschnitt weiter erläutert. "Azure NetApp Files Service-Level".

Die Verarbeitungszeit auf einer GPU betrug 12 Stunden und 45 Minuten. Die Verarbeitungszeit von drei GPUs auf drei Nodes betrug etwa 4 Stunden und 30 Minuten.

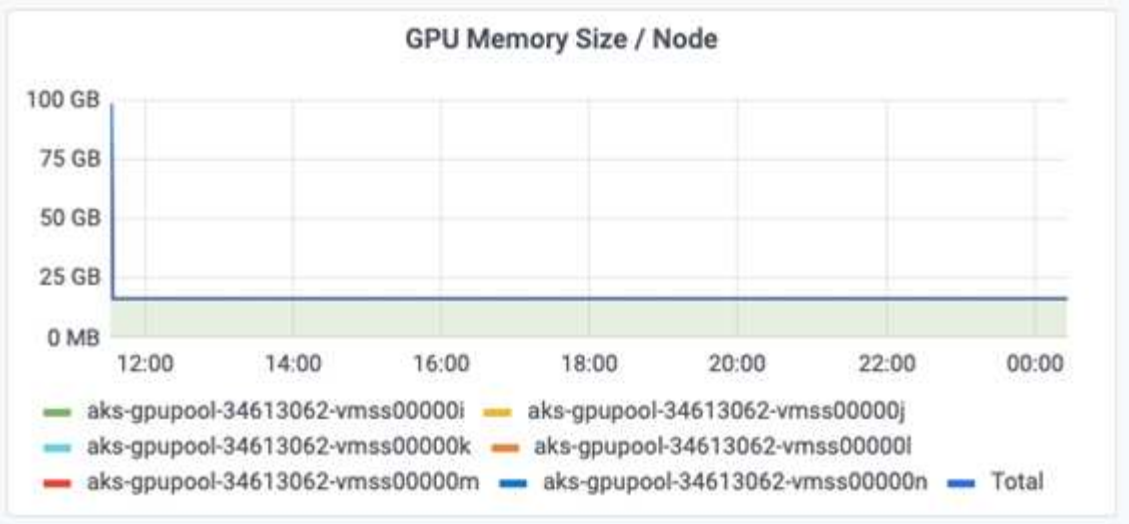
Die im verbleibenden Teil dieses Dokuments veranschaulichen Beispiele für Performance und Skalierbarkeit basierend auf den individuellen Geschäftsanforderungen.

Die Abbildung unten zeigt 1 GPU-Zuweisung und Arbeitsspeicherauslastung.

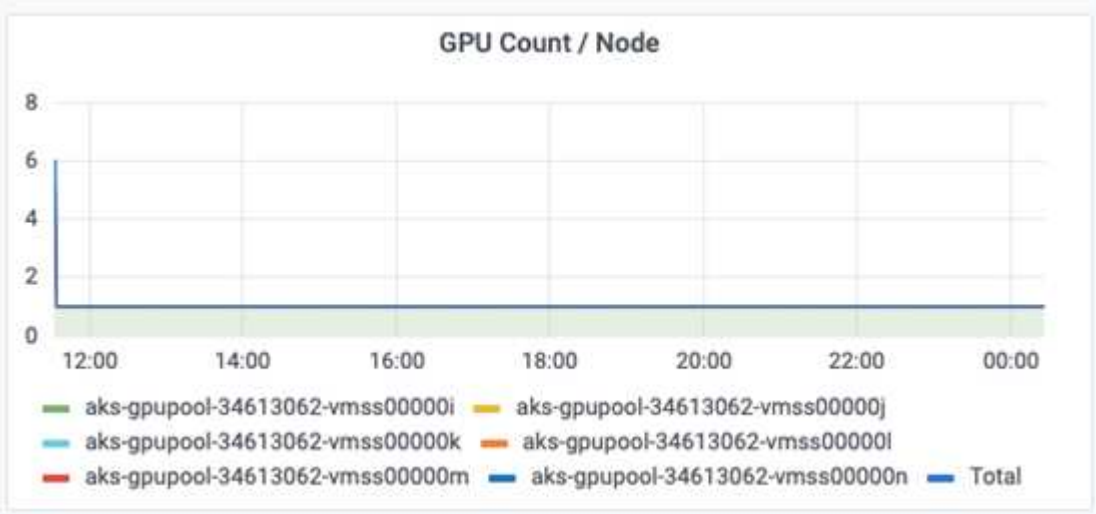
Die Abbildung unten zeigt die GPU-Auslastung mit einem Node.



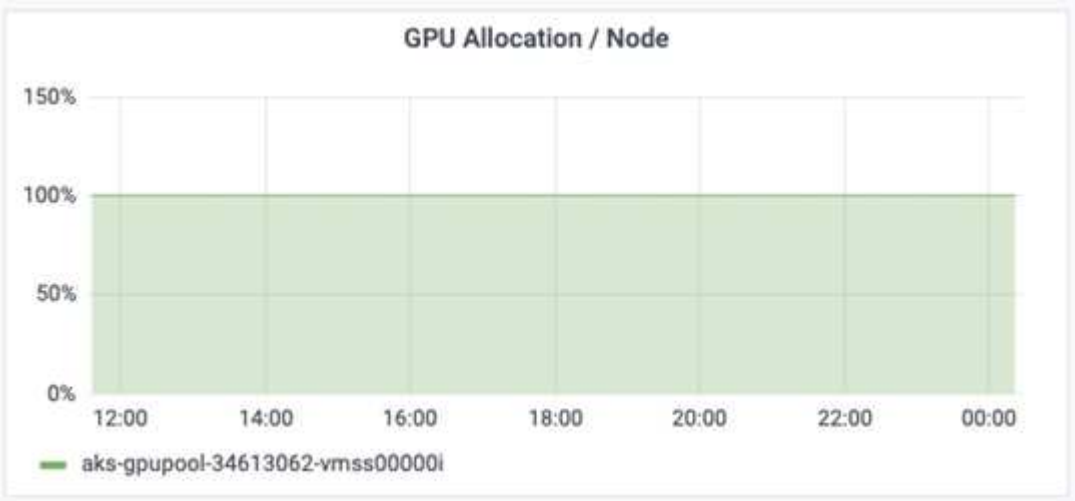
Die Abbildung unten zeigt die Größe des Single-Node-Speichers (16 GB).



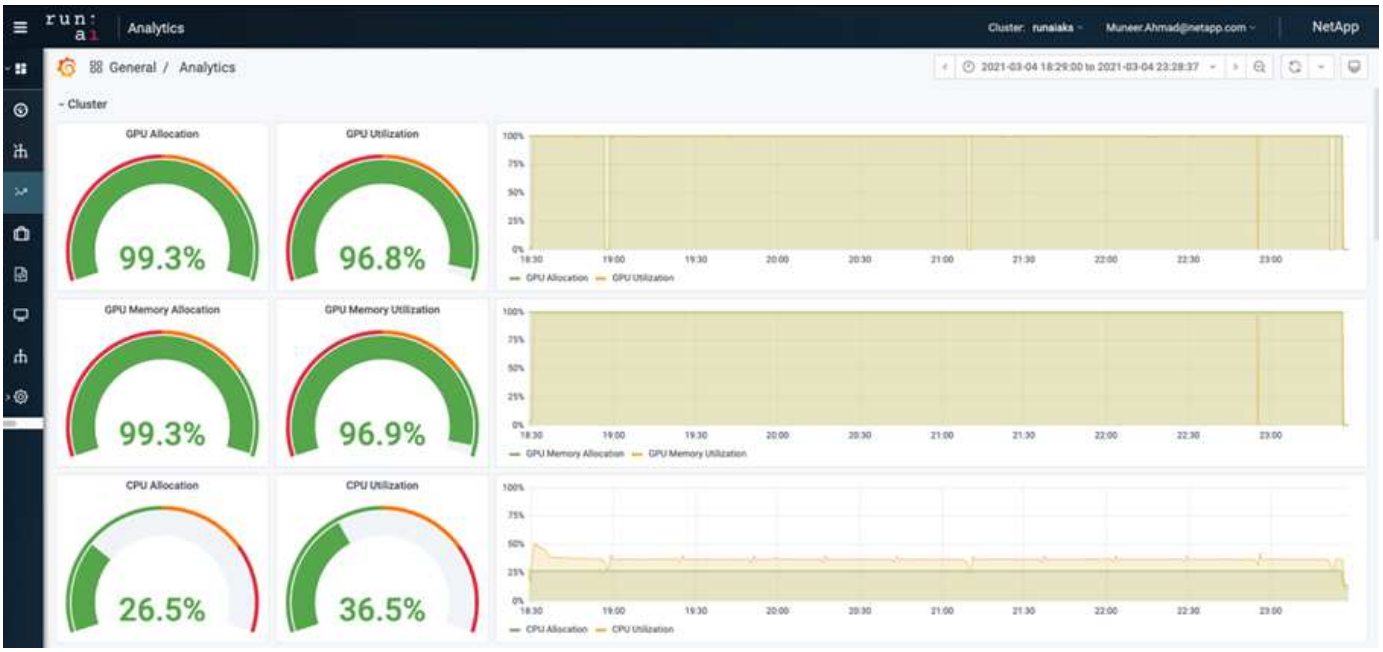
Die Abbildung unten zeigt die GPU-Anzahl einzelner Nodes (1).



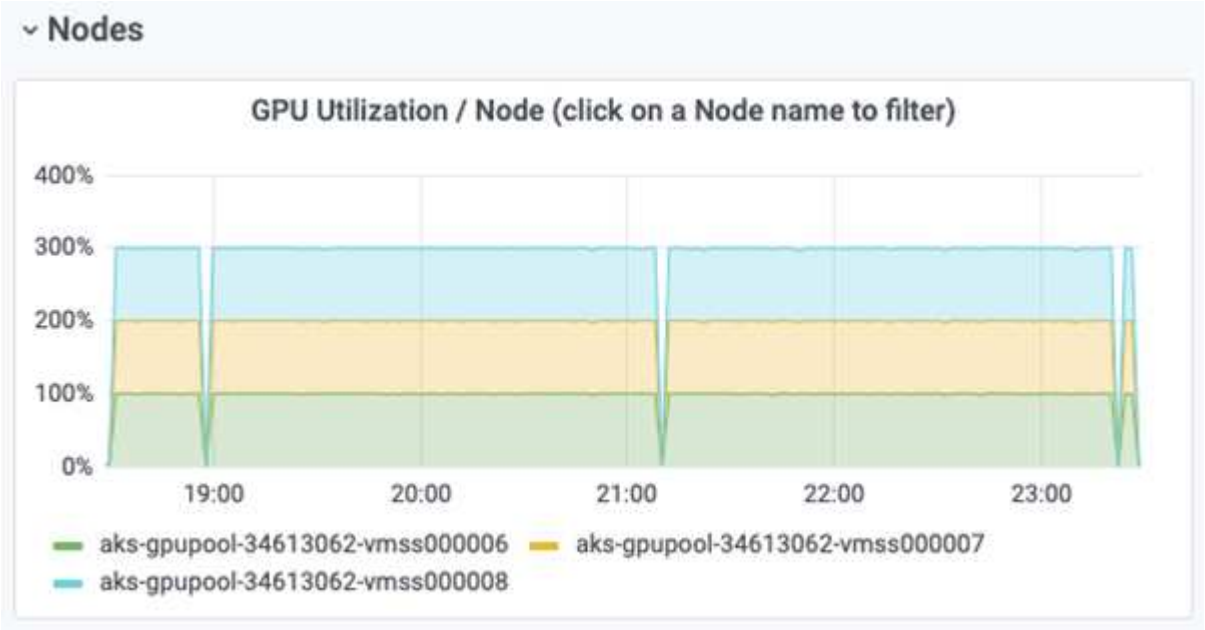
Die Abbildung unten zeigt die GPU-Zuweisung eines einzelnen Node (%).



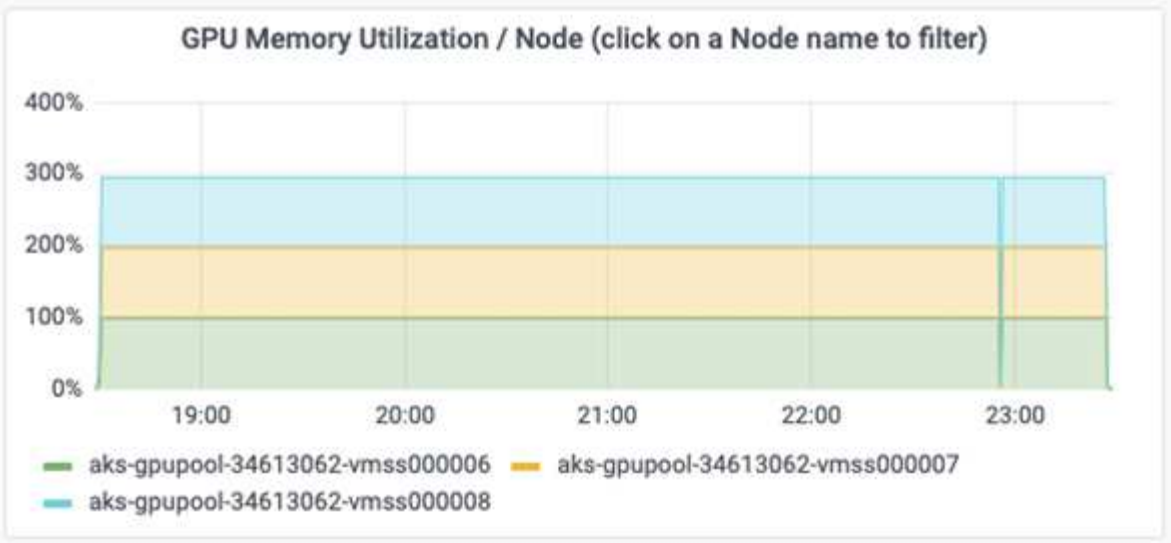
Die Abbildung unten zeigt drei GPUs in drei Nodes: GPU-Zuweisung und Arbeitsspeicher.



Die Abbildung unten zeigt drei GPUs für eine Auslastung von drei Nodes (%).



Die Abbildung unten zeigt drei GPUs über die Speicherauslastung mit drei Nodes (%).



Azure NetApp Files Service-Level

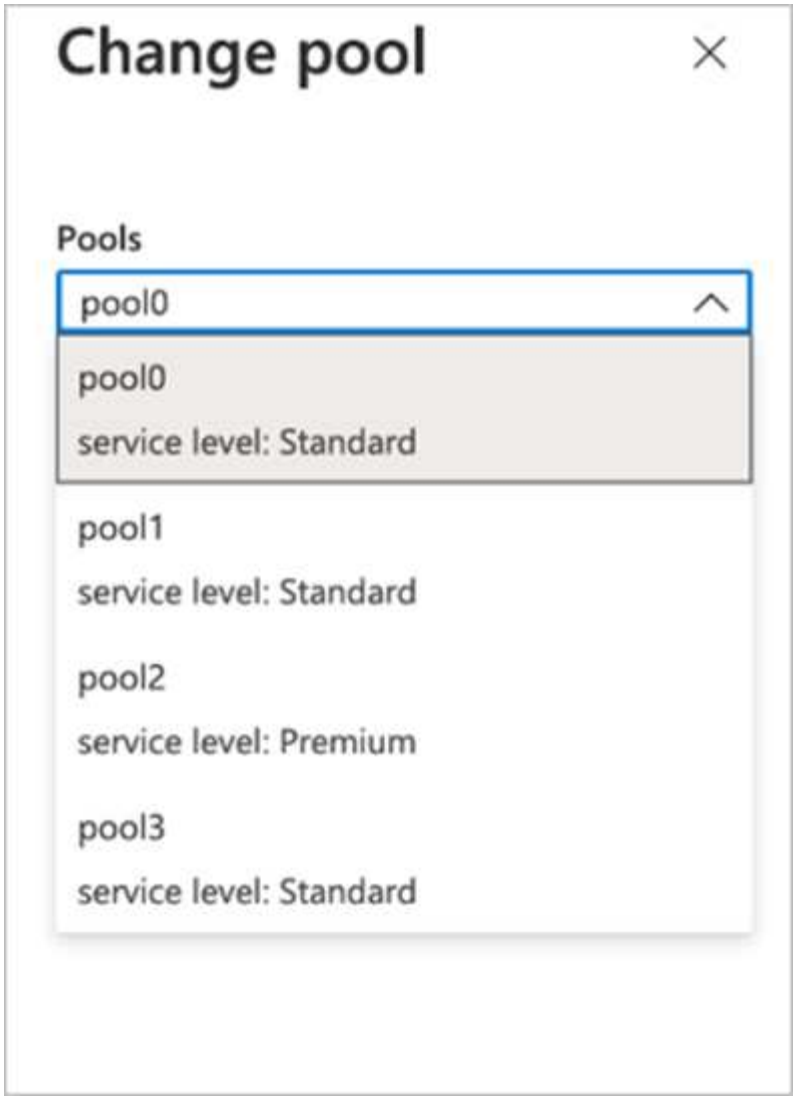
Sie können den Service-Level eines vorhandenen Volumes ändern, indem Sie das Volume in einen anderen Kapazitätspool verschieben, der den verwendet "Service-Level" Sie wollen für das Volume. Bei dieser bestehenden Service-Level-Änderung für das Volume müssen Sie keine Daten migrieren. Er hat auch keinen Einfluss auf den Zugriff auf das Volume.

Profitieren Sie von einer dynamischen Änderung des Service-Levels eines Volumes

Um den Service-Level eines Volumes zu ändern, gehen Sie wie folgt vor:

1. Klicken Sie auf der Seite Volumes mit der rechten Maustaste auf das Volume, dessen Service-Level Sie ändern möchten. Wählen Sie Pool Ändern.

2. Wählen Sie im Fenster Pool ändern den Kapazitätspool aus, in den Sie das Volume verschieben möchten. Klicken Sie anschließend auf OK.



Automatisieren Sie Service Level Change

Die dynamische Änderung des Service-Levels befindet sich derzeit noch in der öffentlichen Vorschau, ist aber standardmäßig nicht aktiviert. Um diese Funktion auf dem Azure-Abonnement zu aktivieren, folgen Sie diesen Schritten im Dokument " "Profitieren Sie von einer dynamischen Änderung des Service-Levels eines Volumes"."

- Für Azure können Sie auch die folgenden Befehle verwenden: CLI. Weitere Informationen zum Ändern der Pool-Größe von Azure NetApp Files finden Sie unter "az netappfiles-Volume: Managt Azure NetApp Files (ANF) Volume-Ressourcen".

```
az netappfiles volume pool-change -g mygroup
--account-name myaccname
-pool-name mypoolname
--name myvolname
--new-pool-resource-id mynewresourceid
```

- Der `set- aznetappfilesvolumepool` Cmdlet, das hier angezeigt wird, kann den Pool eines Azure NetApp Files Volume ändern. Weitere Informationen zum Ändern der Volume-Pool-Größe und Azure

PowerShell finden Sie unter "Ändern Sie den Pool für ein Azure NetApp Files-Volume".

```
Set-AzNetAppFilesVolumePool
-ResourceGroupName "MyRG"
-AccountName "MyAnfAccount"
-PoolName "MyAnfPool"
-Name "MyAnfVolume"
-NewPoolResourceId 7d6e4069-6c78-6c61-7bf6-c60968e45fbf
```

Schlussfolgerung

NetApp und RUN: KI haben bei der Erstellung dieses technischen Berichts gemeinsam die einzigartigen Funktionen der Azure NetApp Files zusammen MIT DER RUN-KI-Plattform zur Vereinfachung der Orchestrierung von KI-Workloads demonstriert. Dieser technische Bericht enthält eine Referenzarchitektur zur Optimierung des Prozesses von Daten-Pipelines und Workload-Orchestrierung für Distributed Lane-Erkennungstrainings.

Fazit: Was verteiltes Training nach Maß angeht (insbesondere in einer Public Cloud-Umgebung), ist die Ressourcenorchestrierung und die Storage-Komponente ein wesentlicher Bestandteil der Lösung. Sicherstellen, dass das Datenmanagement nie die Verarbeitung mehrerer GPUs beeinträchtigt, führt daher zu einer optimalen Auslastung der GPU-Zyklen. Damit wird das System für große verteilte Schulungszwecke so kosteneffizient wie möglich.

Mit der Data Fabric von NetApp können Data Scientists und Data Engineers On-Premises- und Cloud-Ressourcen miteinander verbinden, um synchrone Daten zu speichern – ohne manuelle Eingriffe. Mit anderen Worten: Die Data-Fabric-Infrastruktur gestaltet das Management von KI-Workflows, die über mehrere Standorte verteilt sind, reibungslos. Es erleichtert zudem die bedarfsgerechte Datenverfügbarkeit, indem die Daten näher an das Computing übertragen und Analysen, Training und Validierungen durchgeführt werden – wo und wann immer dies erforderlich ist. Dies ermöglicht nicht nur die Datenintegration, sondern auch Schutz und Sicherheit der gesamten Datenpipeline.

Weitere Informationen

Sehen Sie sich die folgenden Dokumente und/oder Websites an, um mehr über die in diesem Dokument beschriebenen Informationen zu erfahren:

- Datensatz: TuSimple

  "https://github.com/TuSimple/tusimple-benchmark/tree/master/doc/lane_detection"

- Deep Learning Network Architecture: Räumliches Convolutional Neural Network

  "https://arxiv.org/abs/1712.06080"

- Distributed Deep Learning Training Framework: Horovod

  "https://horovod.ai/"

- AUSFÜHRUNG: KI-Container-Orchestrierungslösung: RUN: KI-Produkteinführung

"https://docs.run.ai/home/components/"

- AUSFÜHREN: KI-Installationsdokumentation

  "https://docs.run.ai/Administrator/Cluster-Setup/cluster-install/#step-3-install-runai"
  "https://docs.run.ai/Administrator/Researcher-Setup/cli-install/#runai-cli-installation"

- Einsenden von Jobs in AUSFÜHRUNG: AI CLI

  "https://docs.run.ai/Researcher/cli-reference/runai-submit/"

  "https://docs.run.ai/Researcher/cli-reference/runai-submit-mpi/"

- Azure Cloud-Ressourcen als Azure NetApp Files

  "https://docs.microsoft.com/azure/azure-netapp-files/"

- Azure Kubernetes Service

  "https://azure.microsoft.com/services/kubernetes-service/-features"

- Azure VM SKUs

  "https://azure.microsoft.com/services/virtual-machines/"

- Azure VM mit GPU-SKUs

  "https://docs.microsoft.com/azure/virtual-machines/sizes-gpu"

- NetApp Trident

  "https://github.com/NetApp/trident/releases"

- Data Fabric von NetApp

  "https://www.netapp.com/data-fabric/what-is-data-fabric/"

- NetApp Produktdokumentation

  "https://www.netapp.com/support-and-training/documentation/"

Copyright-Informationen

Copyright © 2024 NetApp. Alle Rechte vorbehalten. Gedruckt in den USA. Dieses urheberrechtlich geschützte Dokument darf ohne die vorherige schriftliche Genehmigung des Urheberrechtsinhabers in keiner Form und durch keine Mittel – weder grafische noch elektronische oder mechanische, einschließlich Fotokopieren, Aufnehmen oder Speichern in einem elektronischen Abrufsystem – auch nicht in Teilen, vervielfältigt werden.

Software, die von urheberrechtlich geschütztem NetApp Material abgeleitet wird, unterliegt der folgenden Lizenz und dem folgenden Haftungsausschluss:

DIE VORLIEGENDE SOFTWARE WIRD IN DER VORLIEGENDEN FORM VON NETAPP ZUR VERFÜGUNG GESTELLT, D. H. OHNE JEGLICHE EXPLIZITE ODER IMPLIZITE GEWÄHRLEISTUNG, EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE STILLSCHWEIGENDE GEWÄHRLEISTUNG DER MARKTGÄNGIGKEIT UND EIGNUNG FÜR EINEN BESTIMMTEN ZWECK, DIE HIERMIT AUSGESCHLOSSEN WERDEN. NETAPP ÜBERNIMMT KEINERLEI HAFTUNG FÜR DIREKTE, INDIREKTE, ZUFÄLLIGE, BESONDERE, BEISPIELHAFTE SCHÄDEN ODER FOLGESCHÄDEN (EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE BESCHAFFUNG VON ERSATZWAREN ODER -DIENSTLEISTUNGEN, NUTZUNGS-, DATEN- ODER GEWINNVERLUSTE ODER UNTERBRECHUNG DES GESCHÄFTSBETRIEBS), UNABHÄNGIG DAVON, WIE SIE VERURSACHT WURDEN UND AUF WELCHER HAFTUNGSTHEORIE SIE BERUHEN, OB AUS VERTRAGLICH FESTGELEGTER HAFTUNG, VERSCHULDENSUNABHÄNGIGER HAFTUNG ODER DELIKTSHAFTUNG (EINSCHLIESSLICH FAHRLÄSSIGKEIT ODER AUF ANDEREM WEGE), DIE IN IRGENDEINER WEISE AUS DER NUTZUNG DIESER SOFTWARE RESULTIEREN, SELBST WENN AUF DIE MÖGLICHKEIT DERARTIGER SCHÄDEN HINGEWIESEN WURDE.

NetApp behält sich das Recht vor, die hierin beschriebenen Produkte jederzeit und ohne Vorankündigung zu ändern. NetApp übernimmt keine Verantwortung oder Haftung, die sich aus der Verwendung der hier beschriebenen Produkte ergibt, es sei denn, NetApp hat dem ausdrücklich in schriftlicher Form zugestimmt. Die Verwendung oder der Erwerb dieses Produkts stellt keine Lizenzierung im Rahmen eines Patentrechts, Markenrechts oder eines anderen Rechts an geistigem Eigentum von NetApp dar.

Das in diesem Dokument beschriebene Produkt kann durch ein oder mehrere US-amerikanische Patente, ausländische Patente oder anhängige Patentanmeldungen geschützt sein.

ERLÄUTERUNG ZU „RESTRICTED RIGHTS“: Nutzung, Vervielfältigung oder Offenlegung durch die US-Regierung unterliegt den Einschränkungen gemäß Unterabschnitt (b)(3) der Klausel „Rights in Technical Data – Noncommercial Items“ in DFARS 252.227-7013 (Februar 2014) und FAR 52.227-19 (Dezember 2007).

Die hierin enthaltenen Daten beziehen sich auf ein kommerzielles Produkt und/oder einen kommerziellen Service (wie in FAR 2.101 definiert) und sind Eigentum von NetApp, Inc. Alle technischen Daten und die Computersoftware von NetApp, die unter diesem Vertrag bereitgestellt werden, sind gewerblicher Natur und wurden ausschließlich unter Verwendung privater Mittel entwickelt. Die US-Regierung besitzt eine nicht ausschließliche, nicht übertragbare, nicht unterlizenzierbare, weltweite, limitierte unwiderrufliche Lizenz zur Nutzung der Daten nur in Verbindung mit und zur Unterstützung des Vertrags der US-Regierung, unter dem die Daten bereitgestellt wurden. Sofern in den vorliegenden Bedingungen nicht anders angegeben, dürfen die Daten ohne vorherige schriftliche Genehmigung von NetApp, Inc. nicht verwendet, offengelegt, vervielfältigt, geändert, aufgeführt oder angezeigt werden. Die Lizenzrechte der US-Regierung für das US-Verteidigungsministerium sind auf die in DFARS-Klausel 252.227-7015(b) (Februar 2014) genannten Rechte beschränkt.

Markeninformationen

NETAPP, das NETAPP Logo und die unter http://www.netapp.com/TM aufgeführten Marken sind Marken von NetApp, Inc. Andere Firmen und Produktnamen können Marken der jeweiligen Eigentümer sein.